

1 PyTorch 分布式编程实践

前三节分别从编程模型（集体通信原语）和硬件底座（NVLink、NVSwitch、InfiniBand、NCCL）两个角度建立了分布式训练的理论基础。本节将两者桥接起来：我们进入 PyTorch 代码层面，展示如何通过 `torch.distributed` 库调用底层 NCCL/Gloo 能力，实现 All-Reduce、Reduce-Scatter 与 All-Gather 等核心集体操作，并学习如何测量实际通信带宽，为后续并行策略的选型提供量化依据。

1.1 torch.distributed 基础与进程管理

1.1.1 动机：从理论原语到可执行代码

集体通信原语（All-Reduce、All-Gather 等）描述的是“做什么”，但要在真实 GPU 集群上运行，还需要解决以下工程问题：

1. 如何在多台机器上启动和管理多个进程？
2. 如何让这些进程彼此发现对方并建立通信通道？
3. 如何根据硬件类型（GPU vs. CPU）选择正确的通信后端？
4. 如何在异步执行的多进程环境中保证同步与数据一致性？

PyTorch 通过 `torch.distributed` 模块为上述问题提供了简洁而统一的解决方案。该模块封装了底层通信库（NCCL、Gloo），向用户暴露高层次的集体通信 API，使开发者无需关心数据包如何在 NVLink 或 InfiniBand 上传输。

1.1.2 核心概念：Rank、World Size 与后端选择

在 `torch.distributed` 的语境中，我们沿用集体通信的标准术语：

- **Rank**：当前进程的编号，从 0 开始，范围是 0 到 $N - 1$ 。
- **World Size (N)**：参与分布式训练的进程总数，通常等于 GPU 总数。

`torch.distributed` 支持多种后端，以适配不同的硬件环境：

- **NCCL 后端** (NVIDIA Collective Communications Library): 专为 GPU 设计, 利用 NVLink/NVSwitch/InfiniBand 实现高速集体通信。这是 GPU 分布式训练的首选后端。
- **Gloo 后端**: 由 Facebook 开发的通用集体通信库, 支持 CPU 和 GPU, 但 GPU 性能不如 NCCL。当环境没有 NVIDIA GPU 时 (如在笔记本电脑上演示), Gloo 是可靠的回退方案。

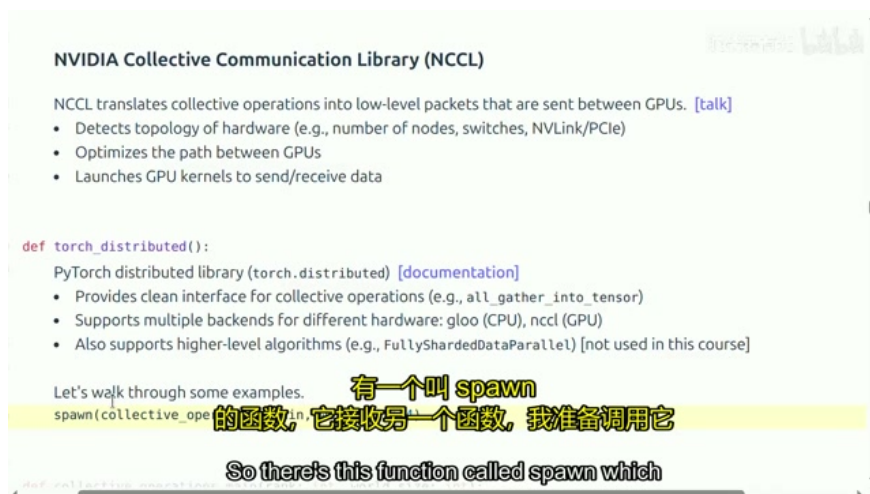


图 1: `torch.distributed` 的功能定位: 为集体通信提供简洁接口, 支持 NCCL (GPU) 和 Gloo (CPU) 后端。

此外, PyTorch 还内置了更高层次的封装, 如 `FullyShardedDataParallel` (FSDP), 但本课程选择从零开始手写分布式代码, 以便深入理解底层机制。

1.1.3 进程启动: `spawn` 函数

在多线程分布式程序中, 通常需要为每块 GPU 创建一个独立的进程。PyTorch 提供了 `torch.multiprocessing.spawn` 函数来完成这一任务:

Listing 1: 使用 `spawn` 启动多线程分布式程序

```
import torch.multiprocessing as mp
```

⁰ 视频截图来源: 00:36:50。

```
def collective_operations_main(rank: int, world_size: int):
    """ 该函数在每个进程上异步执行 (rank=0, ..., world_size-1) """
    setup(rank, world_size)    # 初始化分布式环境
    # ... 执行集体通信操作 ...
    cleanup()                  # 清理分布式资源

if __name__ == "__main__":
    world_size = 4
    mp.spawn(collective_operations_main, args=(world_size,), nprocs=world_size)
```

`spawn` 接收一个目标函数（此处为 `collective_operations_main`），并将其复制 `world_size` 份，分别在独立的进程中运行。每个进程会被分配一个唯一的 `rank` 参数，范围从 0 到 $N-1$ 。这些进程是异步执行的——我们无法预测哪个进程会先完成，它们的完成顺序由操作系统调度器和硬件状态共同决定。

`spawn` 与单进程演示模式 在讲座的逐步讲解中，讲师使用了一个特殊的单进程演示模式（通过 `python -m edtrace.execute -m lecture_07` 运行），以便在无法使用多进程的环境中展示代码逻辑。实际训练时应使用 `torch.multiprocessing.spawn` 或 `torchrun` 启动真正的多进程。

1.1.4 初始化: `init_process_group`

每个进程启动后，第一步是调用 `init_process_group` 来初始化分布式环境：

Listing 2: 初始化分布式进程组

```
import os
import torch.distributed as dist

def setup(rank: int, world_size: int):
    """ 初始化分布式环境（在每个进程开始时调用） """
    # 配置主节点地址和端口，用于进程间协调
    os.environ["MASTER_ADDR"] = "localhost"
```

```

os.environ["MASTER_PORT"] = "15623"

if torch.cuda.is_available():
    # GPU环境：使用NCCL后端
    dist.init_process_group("nccl", rank=rank, world_size=world_size)
else:
    # CPU环境：使用Gloo后端
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

```

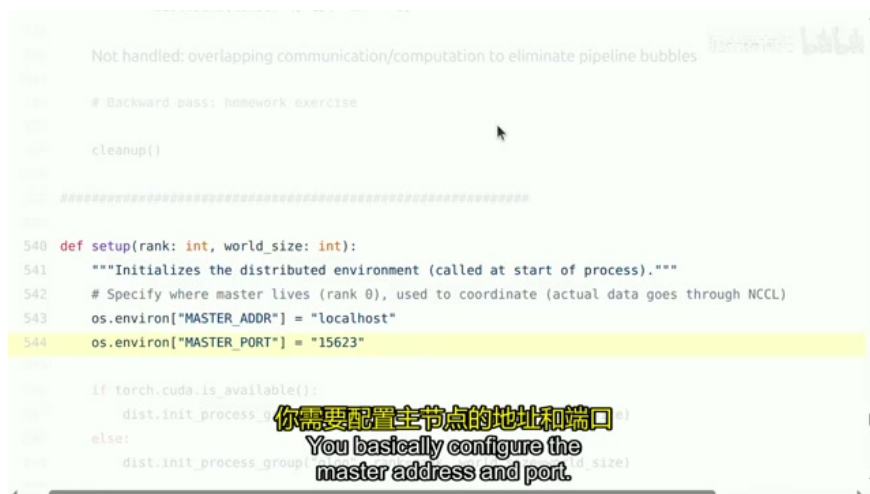


图 2: 分布式环境初始化代码: 配置主节点地址和端口, 根据硬件选择 NCCL 或 Gloo 后端。

需要特别注意的是, `MASTER_ADDR` 和 `MASTER_PORT` 仅用于进程间的元数据协调与同步 (如建立连接、交换地址信息), 实际的数据传输并不经过这个 **TCP 通道**。真正的数据通信由 NCCL 后端通过 NVLink 或 InfiniBand 完成, 否则速度将极其缓慢。

1.1.5 同步屏障: `barrier` 的作用与开销

由于各进程异步执行, 我们无法控制哪个进程先到达某一行代码。如果希望确保某些操作在其他操作之前完成, 就需要插入**同步屏障** (`barrier`):

⁰视频截图来源: 00:38:43。

Listing 3: 同步屏障的使用

`dist.barrier()` # 等待所有进程都到达这一点

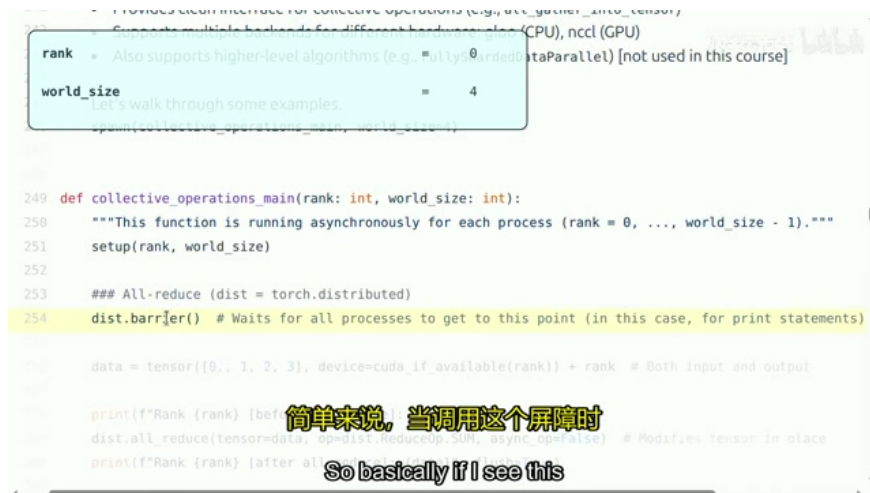


图 3: 代码中的 `dist.barrier()` 调用: 确保所有进程在继续执行前到达同步点。

`barrier` 的语义很简单: 当任一进程调用 `barrier` 时, 它会阻塞并等待, 直到所有进程都调用了 `barrier`, 然后所有进程同时继续执行。这在以下场景中至关重要:

- 在打印调试信息前, 确保所有进程都已准备好相关数据, 避免输出交错混乱。
- 在计时基准测试前, 确保所有前置操作 (如预热) 已完成。
- 在参数更新前, 确保所有进程的梯度同步已完成。

然而, 过多的 `barrier` 也会带来不必要的等待开销——最快的进程必须等待最慢的进程。因此, 在实际训练中应谨慎使用屏障, 尽量让计算与通信重叠, 而非频繁同步。

⁰ 视频截图来源: 00:39:16。

1.2 All-Reduce、Reduce-Scatter 与 All-Gather 代码实现

1.2.1 All-Reduce：就地求和并广播

动机：在数据并行训练中，每个 GPU 计算出的梯度需要汇总求和，然后广播给所有 GPU，以确保各进程的参数更新保持一致。All-Reduce 正是完成这一任务的原语。

代码实现：

Listing 4: All-Reduce 操作示例

```
#### All-Reduce (dist = torch.distributed)
dist.barrier() # 等待所有进程到达此点（确保打印顺序可控）

# 每个rank创建不同的输入数据：rank 0 得到 [0,1,2,3], rank 1 得到 [1,2,3,4],
data = tensor([0., 1, 2, 3], device=cuda_if_available(rank)) + rank

print(f"Rank_{rank} [before_all-reduce]: {data}", flush=True)

# 执行 All-Reduce：就地求和，async_op=False 表示同步执行
dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)

print(f"Rank_{rank} [after_all-reduce]: {data}", flush=True)
```

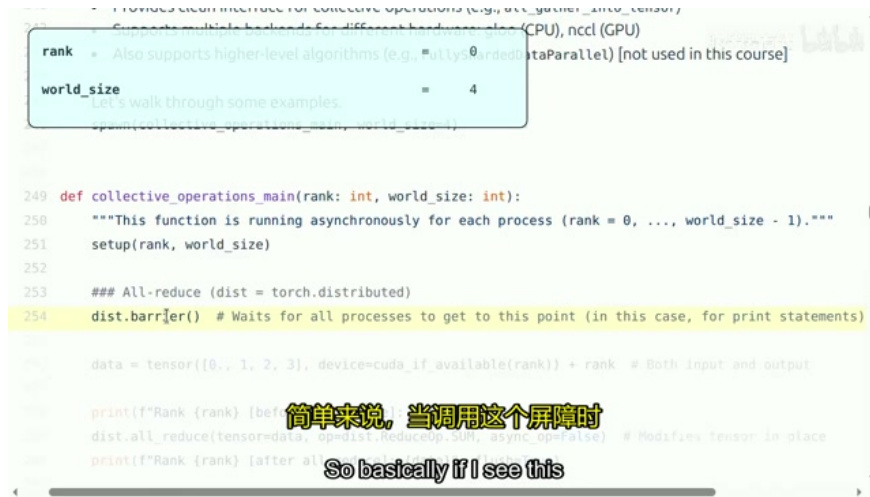


图 4: All-Reduce 代码示例：每个 rank 创建不同的输入张量，执行求和规约后所有 rank 获得相同结果。

执行结果分析：假设 `world_size = 4`，All-Reduce 前的输入为：

Rank 0 : [0, 1, 2, 3]

Rank 1 : [1, 2, 3, 4]

Rank 2 : [2, 3, 4, 5]

Rank 3 : [3, 4, 5, 6]

对每列求和：第 0 列 $0 + 1 + 2 + 3 = 6$ ，第 1 列 $1 + 2 + 3 + 4 = 10$ ，第 2 列 $2 + 3 + 4 + 5 = 14$ ，第 3 列 $3 + 4 + 5 + 6 = 18$ 。All-Reduce 后，每个 Rank 都持有完整的结果：

Rank 0-3 : [6, 10, 14, 18]

⁰ 视频截图来源：00:40:26。

```

Rank 0 [before reduce-scatter]: input = tensor([0., 1., 2., 3.], device='cuda:0'), output = tensor([0.], device='cuda:0')
Rank 1 [after reduce-scatter]: input = tensor([1., 2., 3., 4.], device='cuda:1'), output = tensor([10.], device='cuda:1')
Rank 3 [after reduce-scatter]: input = tensor([3., 4., 5., 6.], device='cuda:3'), output = tensor([18.], device='cuda:3')
Rank 2 [after reduce-scatter]: input = tensor([2., 3., 4., 5.], device='cuda:2'), output = tensor([14.], device='cuda:2')
Rank 0 [before all-gather]: input = tensor([6.], device='cuda:0'), output = tensor([ 0., 10., 14., 18.], device='cuda:0')
Rank 2 [before all-gather]: input = tensor([14.], device='cuda:2'), output = tensor([ 2.,  3.,  4.,  5.], device='cuda:2')
Rank 1 [before all-gather]: input = tensor([10.], device='cuda:1'), output = tensor([ 1.,  2.,  3.,  4.], device='cuda:1')
Rank 3 [before all-gather]: input = tensor([18.], device='cuda:3'), output = tensor([ 3.,  4.,  5.,  6.], device='cuda:3')
Rank 0 [after all-gather]: input = tensor([6.], device='cuda:0'), output = tensor([ 6., 10., 14., 18.], device='cuda:0')
Rank 2 [after all-gather]: input = tensor([14.], device='cuda:2'), output = tensor([ 6., 10., 14., 18.], device='cuda:2')
Rank 1 [after all-gather]: input = tensor([10.], device='cuda:1'), output = tensor([ 6., 10., 14., 18.], device='cuda:1')
Rank 3 [after all-gather]: input = tensor([18.], device='cuda:3'), output = tensor([ 6., 10., 14., 18.], device='cuda:3')
[all_reduce] Rank 0: all_reduce(world_size=4, num_elements=104857600) took 1.60ms
[all_reduce] Rank 2: all_reduce(world_size=4, num_elements=104857600) took 1.38ms
[all_reduce] Rank 1: all_reduce(world_size=4, num_elements=104857600) took 1.50ms
[all_reduce] Rank 3: all_reduce(world_size=4, num_elements=104857600) took 1.38ms
[all_reduce] Rank 1: all_reduce measured bandwidth = 390 GB/s
[all_reduce] Rank 2: all_reduce measured bandwidth = 426 GB/s
[all_reduce] Rank 0: all_reduce measured bandwidth = 366 GB/s
[all_reduce] Rank 3: all_reduce measured bandwidth = 425 GB/s
[reduce_scatter] Rank 0: reduce_scatter(world_size=4, num_elements=104857600) took 2.61ms
[reduce_scatter] Rank 1: reduce_scatter(world_size=4, num_elements=104857600) took 2.47ms
[reduce_scatter] Rank 2: reduce_scatter(world_size=4, num_elements=104857600) took 2.39ms
[reduce_scatter] Rank 3: reduce_scatter(world_size=4, num_elements=104857600) took 2.39ms
[reduce_scatter] Rank 1: reduce_scatter measured bandwidth = 475 GB/s
[reduce_scatter] Rank 0: reduce_scatter measured bandwidth = 490 GB/s
[reduce_scatter] Rank 2: reduce_scatter measured bandwidth = 490 GB/s
[reduce_scatter] Rank 3: reduce_scatter measured bandwidth = 490 GB/s

```

图 5: 终端输出: 展示了 All-Reduce、Reduce-Scatter 和 All-Gather 在各 rank 上的输入输出结果, 以及基准测试的耗时与带宽测量值。

注意, 打印语句的输出顺序由硬件异步执行随机决定, 但数据内容始终正确。

`dist.all_reduce` 的关键参数说明:

- **tensor:** 输入输出张量 (就地修改)。
- **op:** 规约操作类型, 如 `dist.ReduceOp.SUM` (求和)、`AVG` (平均, 部分版本支持)。
- **async_op:** 是否异步执行。False 表示阻塞直到操作完成; True 表示立即返回, 允许计算与通信重叠。

1.2.2 Reduce-Scatter: 输入输出分离的规约散播

动机: 与 All-Reduce 不同, Reduce-Scatter 将输入和输出分离——输入张量保持不变, 规约结果写入独立的输出张量, 且每个 rank 只获得规约结果的一个分片。这在显存优化场景 (如 ZeRO/FSDP) 中非常有用。

代码实现:

Listing 5: Reduce-Scatter 操作示例

⁰ 视频截图来源: 00:43:37。


```

dist.barrier()

# 输入：每个rank持有一个长度为 world_size 的向量
input = torch.arange(world_size, device=cuda_if_available(rank)) + rank
# 输出：预先分配空间
output = torch.empty(1, device=cuda_if_available(rank))

print(f"Rank{rank}[before_reduce-scatter]: input={input}, output={output}")

# 执行 Reduce-Scatter：输入保持不变，输出写入规约结果的分片
dist.reduce_scatter_tensor(output=output, input=input,
                           op=dist.ReduceOp.SUM, async_op=False)

print(f"Rank{rank}[after_reduce-scatter]: input={input}, output={output}")

```

执行结果分析：输入与 All-Reduce 相同。Reduce-Scatter 对每列求和后，将结果分散到不同 rank：

```

Rank 0: output = [6] （第 0 列之和）
Rank 1: output = [10] （第 1 列之和）
Rank 2: output = [14] （第 2 列之和）
Rank 3: output = [18] （第 3 列之和）

```

输入张量 `input` 保持不变，输出张量 `output` 仅包含一个标量值——这是 Reduce-Scatter 与 All-Reduce 的核心区别。

1.2.3 All-Gather：收集分散结果

动机：在 Reduce-Scatter 之后，每个 rank 只持有规约结果的一个分片。如果后续计算需要完整的规约结果，就需要通过 All-Gather 将各 rank 上的分片收集起来。

代码实现：

Listing 6: All-Gather 操作示例

```

# 使用 Reduce-Scatter 的输出作为 All-Gather 的输入
input = output # 每个rank持有一个标量：rank 0 有 [6]，rank 1 有 [10]，...

```

```

output = torch.empty(world_size, device=cuda_if_available(rank))
# 分配输出空间

print(f"Rank {rank} [before all-gather]: input={input}, output={output}")

dist.all_gather_into_tensor(output_tensor=output, input_tensor=input, async_op=

print(f"Rank {rank} [after all-gather]: input={input}, output={output}",
    执行结果: All-Gather 后, 每个 rank 的 output 都包含完整的拼接结果 [6, 10, 14, 18]。

```

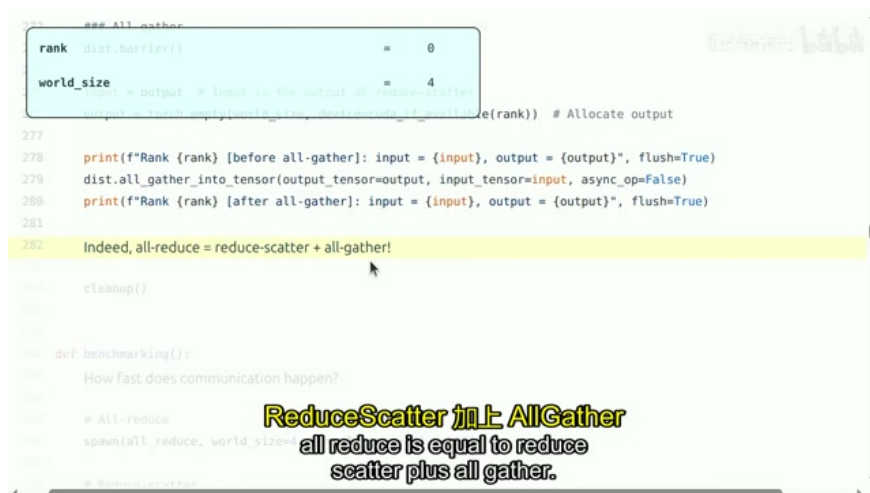


图 6: 代码中明确标注的等价关系: $\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}$ 。

1.2.4 核心等价关系的代码验证

讲座中明确给出了以下等价关系:

$$\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather} \quad (1)$$

这一等价关系不仅在理论层面成立, 上述代码也直接验证了它: 先执行

⁰视频截图来源: 00:45:04。

Reduce-Scatter 将规约结果分散到各 rank，再执行 All-Gather 将这些分散的分片收集并广播到所有 rank，最终效果与直接执行 All-Reduce 完全一致。

为什么需要拆分 All-Reduce? 虽然 NCCL 对 All-Reduce 有专门优化的实现，但在高级技术如 ZeRO/FSDP 中，显式地将 All-Reduce 拆分为 Reduce-Scatter 和 All-Gather 两个独立步骤，可以在中间插入额外的优化逻辑——例如梯度压缩、量化、或计算与通信的重叠调度——从而获得更大的灵活性和性能提升。

1.3 异步操作与有效带宽基准测试

1.3.1 异步集体通信：计算与通信重叠

在前面的示例中，我们将 `async_op` 设为 `False`，这意味着集体通信操作会阻塞直到完成。然而，在实际训练中，这种同步等待会浪费宝贵的 GPU 计算时间。

PyTorch 支持异步集体通信：

Listing 7: 异步 All-Reduce

```
# 启动异步 All-Reduce，立即返回
handle = dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=True)

# 在通信进行的同时，执行其他独立的计算任务
# 例如：加载下一批次数据、执行不依赖当前张量的计算等

# 当需要确保通信完成时，调用等待操作
handle.wait()
# 或者使用 barrier 进行同步
dist.barrier()
```


异步操作中的同步陷阱 讲座中有一个学生提问极具启发性：“如果先调用 `barrier`，但此时 CUDA 内核还没算完，会发生什么？”讲师指出，这种情况下 `barrier` 实际上并没有真正同步——因为 `barrier` 只同步进程到达的状态，而不等待 CUDA 内核完成。正确的做法是先调用 `torch.cuda.synchronize()` 等待 CUDA 内核完成，再调用 `dist.barrier()` 等待所有进程到达同步点。顺序错误会导致计时结果不准确，甚至产生数据竞争。

1.3.2 基准测试方法：预热、计时与同步

要准确测量集体通信的性能，必须遵循严格的基准测试流程，以消除两种异步带来的干扰：

1. **CUDA 内核异步**：CUDA 操作默认异步执行，调用返回不代表内核已完成。
2. **多进程异步**：各进程独立运行，完成时间可能不同。

Listing 8: All-Reduce 基准测试代码

```
def all_reduce(rank: int, world_size: int, num_elements: int):
    setup(rank, world_size)

    # 创建张量
    data = torch.randn(num_elements, device=cuda_if_available(rank))

    # 预热：确保 GPU 已就绪
    dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
    torch.cuda.synchronize() # 等待 CUDA 内核完成
    dist.barrier()           # 等待所有进程到达此点

    # 正式计时
    start_time = time.time()
    dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
    torch.cuda.synchronize() # 等待 CUDA 内核完成
```

```

dist.barrier()          # 等待所有进程到达此点
end_time = time.time()

duration = end_time - start_time
print(f"[all_reduce] Rank {rank}: world_size={world_size}, "
      f"num_elements={num_elements} took {render_duration(duration)}", f

```

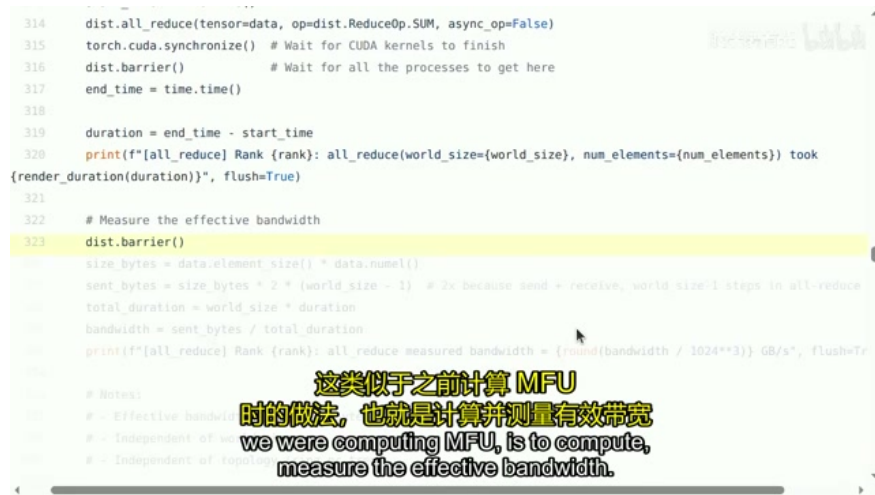


图 8: All-Reduce 基准测试代码：包含预热、`cuda.synchronize` 同步、`barrier` 同步和精确计时。

预热阶段的目的是让 GPU 进入稳定工作状态，避免首次启动内核的冷启动开销影响测量结果。计时阶段前后各放置一对 `torch.cuda.synchronize()` 和 `dist.barrier()`，确保计时区间内只包含 All-Reduce 操作的实际耗时。

1.3.3 有效带宽的计算

测量到耗时后，下一个问题是：这个速度到底快还是慢？为了回答这个问题，我们需要计算有效带宽（Effective Bandwidth）。

核心思路：计算在整个通信过程中实际传输了多少字节，然后用总字节数除以总时间，得到有效带宽。

⁰视频截图来源：00:47:14。

```

dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
torch.cuda.synchronize() # Wait for CUDA kernels to finish
dist.barrier() # Wait for all the processes to get here
end_time = time.time()

duration = end_time - start_time
print(f"[all_reduce] Rank {rank}: all_reduce(world_size={world_size}, num_elements={num_elements}) took {duration} s")

# Measure the effective bandwidth
dist.barrier()
size_bytes = data.element_size() * data.numel()
sent_bytes = size_bytes * 2 * (world_size - 1) # 2x because send + receive, world_size-1 steps in all-reduce
total_duration = world_size * duration
bandwidth = sent_bytes / total_duration
print(f"[all_reduce] Rank {rank}: all_reduce measured bandwidth = {round(bandwidth / 1024**3)} GB/s", flush=True)

# Notes:
# - Effective bandwidth ~ 2
# - Independent of world size
# - Independent of topology (ring)

```

而带宽就是发送的总字节数除以总时间
and the bandwidth is the bytes
sent divided by the total duration.

图 9: 有效带宽计算公式: $\text{sent_bytes} = \text{size_bytes} * 2 * (\text{world_size} - 1)$, 考虑了发送与接收双向数据流。

对于 All-Reduce 操作, 有效带宽的计算公式为:

$$\text{sent_bytes} = \underbrace{\text{size_bytes}}_{\text{张量总字节数}} \times \underbrace{2}_{\text{发送 + 接收}} \times \underbrace{(N - 1)}_{\text{迭代次数}} \quad (2)$$

其中:

- $\text{size_bytes} = \text{element_size} \times \text{num_elements}$, 即张量的总字节数。
- 因子 2 表示每次迭代中既需要发送数据, 又需要接收规约后的结果。
- 因子 $(N - 1)$ 表示在 All-Reduce 的朴素实现中, 需要进行 $N - 1$ 次迭代来完成所有进程的规约。

总持续时间需要考虑所有进程:

$$\text{total_duration} = N \times \text{duration} \quad (3)$$

最终带宽为:

$$\text{BW} = \frac{\text{sent_bytes}}{\text{total_duration}} = \frac{\text{size_bytes} \times 2 \times (N - 1)}{N \times \text{duration}} \quad (4)$$

⁰视频截图来源: 00:50:01。

当 N 较大时, $(N - 1)/N \approx 1$, 公式简化为:

$$BW \approx \frac{2 \times \text{size_bytes}}{\text{duration}} \quad (5)$$

有效带宽的关键性质

1. **与进程数无关:** 当 N 增大时, $(N - 1)/N \rightarrow 1$, 带宽趋近于 $2 \times \text{size_bytes}/\text{duration}$, 与 N 无关。
2. **与网络拓扑无关:** 无论 NCCL 选择环形拓扑 (ring) 还是树形拓扑 (tree) 来实现 All-Reduce, 最终测得的有效带宽都由 NCCL 的优化策略决定, 对用户透明。

在讲座的实测中, 对约 1 亿个元素执行 All-Reduce, 测得的有效带宽约为 **400 GB/s**。这一数值与 NVLink 的理论带宽 (约 1.8 TB/s) 相比仍有差距, 原因是 All-Reduce 涉及多次数据搬移和规约计算, 实际效率受多种因素影响。

1.3.4 Reduce-Scatter 的带宽计算

Reduce-Scatter 的带宽计算与 All-Reduce 非常相似, 但有一个关键区别——没有因子 2:

Listing 9: Reduce-Scatter 带宽计算

```
# 输入数据的总字节数
data_bytes = input.element_size() * input.numel()

# 实际发送的字节数 (没有 2 倍系数)
sent_bytes = data_bytes * (world_size - 1)

# 总持续时间
total_duration = world_size * duration

# 带宽
bandwidth = sent_bytes / total_duration
```

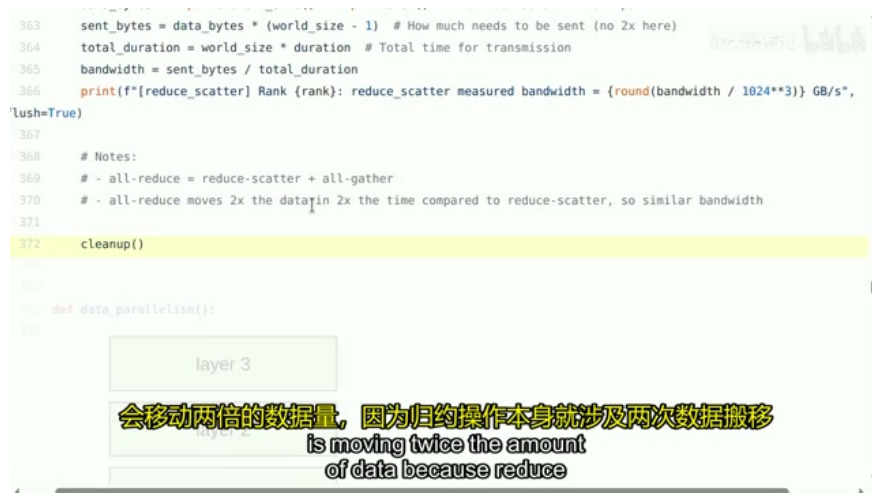



图 10: Reduce-Scatter 基准测试与带宽计算代码：注意 `sent_bytes` 没有 $2\times$ 系数。

为什么没有因子 2？因为 Reduce-Scatter 只涉及单向的数据搬移——将输入数据发送给其他进程进行规约，而不需要像 All-Reduce 那样再把结果广播回来。

1.3.5 All-Reduce 与 Reduce-Scatter 的带宽对比

讲座最后总结了两者的关系：

$$\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather} \quad (6)$$

这意味着：

- All-Reduce 需要移动**两倍**的数据量（Reduce-Scatter 搬一次，All-Gather 再搬一次）。
- All-Reduce 花费**两倍**的时间。
- 但由于数据量和时间同比例增加，**有效带宽保持不变**。

实测结果验证了这一结论：All-Reduce 和 Reduce-Scatter 测得的带宽都在 **400 GB/s** 左右，略有波动，但基本处于同一量级。

⁰视频截图来源：00:52:08。

带宽测量的核心启示

1. 有效带宽是衡量分布式通信性能的核心指标，计算方式为 $BW = \text{sent_bytes} / \text{total_duration}$ 。
2. All-Reduce 的带宽公式包含 $2\times$ 系数，反映其双向数据搬移特性；Reduce-Scatter 没有该系数。
3. 带宽与进程数 N 和网络拓扑无关（由 NCCL 自动优化），这简化了性能预测和硬件选型。
4. 基准测试时必须严格同步（`cuda.synchronize + barrier`），否则测量结果会包含未完成的异步操作，导致严重偏差。

1.4 本章小结

本节从 PyTorch 代码层面深入实践了分布式集体通信，建立了从 API 调用到底层硬件执行的完整认知链条：

1. **torch.distributed 架构**：提供简洁的集体通信 API，支持 NCCL（GPU）和 Gloo（CPU）后端，将高层操作映射到底层通信库。
2. **进程管理**：使用 `spawn` 启动多进程，每个进程通过 `init_process_group` 初始化，配置 `MASTER_ADDR/PORT` 用于协调，实际数据传输由 NCCL 通过 NVLink/InfiniBand 完成。
3. **同步机制**：`barrier` 用于进程级同步，`torch.cuda.synchronize` 用于 CUDA 内核级同步。两者缺一不可，顺序错误会导致伪同步和数据竞争。
4. **三大核心操作**：
 - **All-Reduce**：就地求和并广播，是数据并行梯度同步的主力。
 - **Reduce-Scatter**：输入输出分离，逐维度规约后分散结果，避免单点显存爆炸。
 - **All-Gather**：收集分散的分片，让每个 rank 获得完整结果。

5. **核心等价关系:** $\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather}$ 。在 ZeRO/FSDP 等高级技术中，显式拆分这一等式可以获得更大的优化灵活性。
6. **异步通信:** 通过 `async_op=True` 实现计算与通信重叠，是提升分布式训练效率的关键技术。
7. **带宽基准测试:** 严格遵循“预热 $\rightarrow$ 同步 $\rightarrow$ 计时 $\rightarrow$ 同步”的流程，计算有效带宽。All-Reduce 带宽 $\approx 2 \times \text{size_bytes} / \text{duration}$ ，与进程数和拓扑无关。

掌握了这些编程实践后，我们已经具备了实现和理解三种并行训练策略（数据并行、张量并行、流水线并行）的代码能力。下一节将把这些基础构建块组合起来，系统讲解如何在 PyTorch 中实现每一种并行策略，并分析其通信开销、适用场景与硬件要求。